Shailesh Yadav-da

Data Analyst

PANDAS — Complete Course to Transform Your Data Skills

Welcome To My Post

Scroll More

🔰 LEVEL 1: PANDAS BEGINNER (FOUNDATION)

What is Pandas?

Pandas is a Python library used for:

- Data Analysis
- Data Cleaning
- Data Manipulation
- Working with CSV, Excel, SQL data
-

Core Pandas objects:

- Series (1D)
- DataFrame (2D table)

Import Pandas

```
import pandas as pd
```

Load CSV File

```
df =
pd.read_csv("employee_data.csv")

print(df.head())
# Shows first 5 rows

print(df.tail())
# Shows last 5 rows
```

Basic Inspection (VERY IMPORTANT)

```
print(df.shape)
# (50, 8)

print(df.columns)
#
Index(['emp_id','name','age','department',
'city','salary','experience','join_date'])

print(df.info())
# Column names, data types, non-
null count

print(df.describe())
# Statistical summary of numeric
columns
```

Selecting Columns

```
print(df["salary"])

# Returns salary column

print(df[["name", "department", "salary"]])

# Multiple columns
```

🔰 LEVEL 2: DATA CLEANING (MOST IMPORTANT)

Handling Missing Values

```
print(df.isnull().sum())

# Shows missing values count per column
```

Fill missing age with mean

```
print(df["age"].fillna(df["age"].mean(), inplace=True))

# Missing ages filled
```

Remove Extra Spaces (TEXT CLEANING)

```
df["name"] = df["name"].str.strip()
df["city"] = df["city"].str.strip()
```

Fix Case Issues

```
df["city"] = df["city"].str.title()
df["department"] = df["department"].str.upper()
```

Change Data Types

```
df["join_date"] = pd.to_datetime(df["join_date"])

# Converted to datetime
```

🔰 LEVEL 3: FILTERING & CONDITIONS

🔟 Single Condition

```
print(df[df["salary"] > 70000])

# Employees with salary above 70000
```

Multiple Conditions

```
print(df[(df["department"] == "IT") &
(df["experience"] >= 5)])

# IT employees with experience ≥ 5
```

isin()

```
print(df[df["city"].isin(["Delhi",
"Mumbai"])])

# Employees from Delhi or Mumbai
```

🔰 LEVEL 4: SORTING & INDEXING

Sorting

```
print(df.sort_values(by="salary", ascending=False))

# Highest salary first
```

Reset / Set Index

```
df.set_index("emp_id", inplace=True)

df.reset_index(inplace=True)
```

🔰 LEVEL 5: GROUPBY

Average Salary by Department

```
print(df.groupby("department")
["salary"].mean())
```

Multiple Aggregations

```
print(df.groupby("department").agg({
  "salary": ["min", "max", "mean"],
  "experience": "mean"
}))
```

Count Employees per City

```
print(df.groupby("city")
["emp_id"].count())
```

unique

```
unique_names = df["city"].unique()
print(unique_names)
```

Output:

['Delhi', 'Mumbai', 'Pune', 'Bangalore']

“Value → kitni baar?” = value_counts()

```
print(df.assign(
city=df['city'].str.title().str.strip()).
groupby
("city")["emp_id"].count())
```

🔰 LEVEL 6: STRING OPERATIONS

```
print(df["name"].str.upper()
# Names in uppercase)

print(df["name"].str.contains("A"))
# Names containing letter A
```

Note:

print(df.groupby("city")["salary"].sum())

```
Bangalore    352000
Delhi       1286000
Mumbai       810000
Pune         741000
```

🔰 LEVEL 8: ADVANCED DATA ANALYSIS

Add a new column

Apply Function

```
def salary_level(sal):
    if sal >= 80000:
        return "High"
    elif sal >= 50000:
        return "Medium"
    else:
        return "Low"

df["salary_level"] =
df["salary"].apply(salary_level)
```

Value Counts

```
print(df["department"].value_counts())
```

```
department
IT         13
HR         13
Sales      12
Finance    12
```

🔰 LEVEL 9: EXPORT CLEAN DATA

Save Cleaned CSV

```
df.to_csv("cleaned_employee_data.csv", index=False)
# Cleaned data saved
```

🐼 How to Add a Column in Pandas

Add Column with Direct Assignment (MOST COMMON)

Example: Add bonus column

```
df["bonus"] = 5000
# A new column 'bonus' added with value 5000 for all rows
```

Add Column Using Calculation

Example: 10% bonus from salary

```
df["bonus"] = df["salary"] * 0.10
# bonus = 10% of salary
```

✅ Delete ONE column (MOST USED)

```
df.drop("bonus", axis=1, inplace=True)
# Column 'bonus' deleted
```

✅ Delete MULTIPLE columns (VERY COMMON)

```
df.drop(["city_upper", "dept_short"], axis=1, inplace=True)
# Multiple columns deleted
```

Rename ONE column

Rename ONE column (MOST ASKED)

```
df.rename(columns={"salary":
"monthly_salary"}, inplace=True)

# salary → monthly_salary
```

Rename MULTIPLE columns

```
df.rename(
    columns={
        "emp_id": "employee_id",
        "join_date": "joining_date"
    },
    inplace=True
)
# Multiple columns renamed
```

Pandas: How to Add a NEW ROW (New Data)

✅ Add ONE row using loc (MOST USED)

```
df.loc[len(df)] = [6, "Rohit", "IT", 60000]

# New row added at the end
```

👉 Use when:

- You know all column values
- Small data insertion

✅ Add row using Dictionary (CLEAN & SAFE)

```
df.loc[len(df)] = {
    "emp_id": 7,
    "name": "Neha",
    "department": "HR",
    "salary": 45000
}
# New row added using column names
```

✅ Add MULTIPLE rows (MOST PROFESSIONAL WAY)

```
new_rows = pd.DataFrame([
   {"emp_id": 8, "name": "Amit", "department": "Sales", "salary": 50000},
   {"emp_id": 9, "name": "Pooja", "department": "IT", "salary": 65000}
])

df = pd.concat([df, new_rows], ignore_index=True)

# Multiple rows added
```

Convert join_date column to DATE (Pandas)

✅ BEST & STANDARD WAY

```
df["join_date"] = pd.to_datetime(df["join_date"])

# join_date converted to datetime
```

✅ If date format is string like: 2022-03-12

```
df["join_date"] = pd.to_datetime(df["join_date"], format="%Y-%m-%d")
# Converted to date
```

✅ CHECK (IMPORTANT)

```
df["join_date"].dtype
# datetime64[ns]
```

🧠 MOST IMPORTANT RULE

Always convert date columns using pd.to_datetime()

Extract Year, Month, Day from join_date

✅ YEAR

```
df["year"] = df["join_date"].dt.year
# 2022
```

✅ MONTH

```
df["month"] =
df["join_date"].dt.month
# 3
```

✅ DAY

```
df["day"] = df["join_date"].dt.day
# 12
```

📌 Pandas: loc & iloc

🔹 loc (MOST USED)

Use when you have:

- Column names
- Conditions

✅ Filter rows (daily use)

```
df.loc[df["salary"] > 70000]

# Rows where salary > 70000
```

✅ Select specific columns with condition

```
print(df.loc[df["salary"] > 100000,
["department","name","salary"]])

# Name & salary of experienced employees
```

🔹 iloc

Use when you have:

- Row numbers
- Column positions

✅ Select rows by position

```
df.iloc[0:5]
# First 5 rows
```

✅ Select specific cell

```
df.iloc[2, 3]
# Value at 3rd row, 4th column
```

📌 Pandas: concat

🔹 What is concat?

concat is used to combine DataFrames (mainly used to append data).

✅ MOST COMMON USE (Row-wise)

Combine two DataFrames (VERY COMMON)

```
pd.concat([df1, df2])

# Rows of df2 added below df1
```

✅ **Column-wise concat (Sometimes used)**

```
pd.concat([df1, df2], axis=1)

# DataFrames joined side by side
```

📌 Pandas: merge & join

🔹 merge (MOST USED)

Used to combine DataFrames like SQL JOIN

✅ Inner Join (DEFAULT & MOST COMMON)

```
print(pd.merge(emp, sal, on="emp_id"))

# Returns matching rows from both tables
```

✅ Left Join (VERY COMMON)

```
print(pd.merge(emp, sal,
on="emp_id", how="left"))

# All rows from emp, matching from sal
```

✅ Right Join (Less common but important)

```
print(pd.merge(emp, sal, on="emp_id",
how="right"))

# All rows from sal, matching from emp
```

✅ Outer Join (Used in analysis)

```
print(pd.merge(emp,sal,on="emp_id",
how="outer"))

# All rows from both tables
```

🔹 join

Used when joining on index

✅ Join using index (COMMON CASE)

```
emp = emp.set_index("emp_id")
sal = sal.set_index("emp_id")

# emp_id is now index in both
```

```
print(emp.join(sal))
```

Name. Shailesh Yadav

GitHub link
github.com/shailesh-yadav-da

site:
https://sites.google.com/view/shailesh-yadav-da/home